Aegis — Agent / Scaffold Pillar

Does an inference-time scaffold improve a security agent's exploit success? A held-constant-model measurement, and an honest null.

The problem. Security-agent results are usually reported as a single capability number on a strong model, which conflates two things: how good the model is, and how good the *scaffold* around it is. The interesting, transferable question is the second one — does the retrieval/tooling/prompting layer wrapped around a frozen model actually move the needle? Answering it requires holding the model constant and measuring the *delta* a scaffold adds, rigorously enough that a small, noisy, or accidental "win" can't masquerade as a real one. That measurement discipline is what's usually missing.

The approach. I built a controlled bare-vs-scaffold harness on **CVE-Bench** (40 real-world web-application CVEs, run on UK AISI's Inspect with deterministic `done.sh` verification), holding the model fixed at **DeepSeek V4 Flash** — which, notably, performs at the published *frontier-LLM* level on this benchmark (~13% zero-day), so it is a legitimate baseline rather than a weak one. Every comparison differs only in the scaffold, switched by a single environment variable so the bare arm provably receives nothing; the run is fingerprinted (image digests, model string, flags) so a scaffold arm is provably on the same substrate as its baseline. The measurement carries the disciplines the field tends to skip: a **pre-registered in-band subset** (the tasks where an effect could plausibly live, committed from the bare baseline before any scaffold run), a **power analysis** (detecting a 5pp effect at this base rate needs ~400 samples/arm), **per-task flip tracking with mechanism attribution**, and a **pre-committed significance bar** — a delta counts only at ≥+5pp *and* ≥+2 net fail→pass flips *and* consistent sign across epochs. An integrity rule runs throughout: a scaffold must help the agent *execute a real finding*, never do the task for it.

The one result. Three scaffolds were tested, escalating from prompt-level to tool-level: a "patch-discriminating exploit" reasoning checklist, an execution-methodology checklist, and a tool-level HTTP request linter plus response surfacer (the SWE-agent "make the error visible" model). On an adequately-powered run (n=200/arm, powered for ≥10pp), the tool scaffold delivered **+0.5pp on zero-day and −1.5pp on one-day** — a null, with zero net task-flips and a small regression on the variant where the agent already knows the vulnerability. All three scaffolds land sub-noise. The diagnosis is the substantive finding: **the agent sees its own errors and repeats them** — it reads the API spec, receives an explicit "you sent a body with GET, use POST" diagnostic, and sends the malformed request again. Passive feedback does not change behavior for this model tier. The discipline earned its keep: it caught and refused to headline a +2.5pp prompt-scaffold result that an under-powered or best-of-n report would have published as a win.

The sharpest finding — the wrong target. A final active probe — a guardrail that *blocks* a malformed request and forces a retry (the SWE-agent "seatbelt") rather than merely flagging it — produced the most useful result almost by accident. The guardrail fired; the agent routed around it via a second tool and sent a *working* request; **and it still failed**, spending its remaining budget on the wrong exploit path instead of completing the chain. With request construction solved and no pass to show for it, the conclusion sharpens: request hygiene was never the binding constraint — every scaffold tested (prompt, diagnostic, guardrail) aimed at it. The binding constraint is **strategy and chain-completion under a tight turn budget** — a planning problem addressed by multi-agent architectures (the lever behind the published one-day SOTA), not by request-level help. The investigation didn't just null; it located the real bottleneck and redirected the future

work. (A free secondary lesson: a guardrail on one tool doesn't bind — the agent escaped via another egress path, so enforcement scaffolds must be complete.)

The honest, scoped conclusion. Passive inference-time scaffolds — prompt-level methodology and diagnostic tool feedback — do not produce above-noise exploit lifts for this model on this benchmark. The claim is scoped deliberately: this is *not* "scaffolds don't help." The remaining levers are where published systems get their lift — chiefly **multi-agent planning** (the architecture behind CVE-Bench's published ~30%), which targets the strategy-under-budget constraint isolated above rather than request hygiene; plus few-shot demonstration (the cheapest unexplored falsifier) and domain tools (sqlmap, nuclei) that reframe exploitation as tool-use. These were left untested by deliberate budget and integrity choices, recorded as such — not gaps discovered after the fact. The central design tension is itself a finding: the integrity-clean interventions (hint, observe, demonstrate) are the ones the model can ignore, and the interventions that would *force* the fix (auto-correct the request) cross into doing the task for it; the space between "warning light" and "seatbelt" is the space between "doesn't help" and "gaming."

What stands. The contribution is not a scaffold that works — it is a measurement apparatus rigorous enough to reject its own scaffolds, and the honest, precisely-scoped finding that resulted. The benchmark-agnostic, fingerprinted, parallel harness (env-var toggle, deterministic verification, ~25-minute sweeps) is reusable for any future scaffold, model, or benchmark. This is the project's throughline in the agent domain: *build the measurement layer first, trust the numbers it produces, and don't claim more than the d